

Data Structures Laboratory Sheet No. 4 Instituto Superior de Engenharia de Coimbra
--

1- `Collections.fill` receives a list (`List`) and a value, and puts this value in all list positions. Implement an equivalent method.

2 - Build a method that prints the contents of a list (`List`) in reverse order using a `ListIterator` properly.

3 – We desire to create a class that implements a stack of generic values, offering the methods `empty ()`, `peek ()`, `pop ()` and `push (value)`. The method `empty ()` indicates that the stack is empty (that is, returns true if it is empty), the `peek ()` method returns the value at the top of the stack, the `pop ()` method returns and also removes the value that is at the top of the stack, while `push (value)` adds a new value to the top of the stack. The objects are stored internally through a list (`List`) which is received as a parameter of the constructor. Implement this class and:

- a) Review the performance of its implementation when using an `ArrayList` to store the values from the stack.
- b) Review the performance of its implementation when using one `LinkedList` to store the values from the stack.
- c) Explain any discrepancy or difference of results between both implementations
- c) In view of the above results, identify what type of list to be used to implement a stack.

4 - Repeat the previous paragraph, this time implementing a `Queue`. This should support the methods `add(e)` and `remove()` and `element ()`. The `add(e)` method adds to the end of the queue, while the latter two methods are concerned with the element at the head of the list - the method `remove ()` removes and returns that element, whereas `element ()` just returns it .

5 - `Collections.reverse` receives a list (`List`) and changes the order of its elements. Implement an equivalent method (of course without resorting to `Collections.reverse` method), ensuring good performance (in terms of complexity) regardless of the type of list that is received (there are several ways to solve this problem correctly - Note that unlike the problem 2 here it is necessary that the list of elements to be effectively exchanged, and that the method does not return a new list, but makes changes to the received list)

6 - Build a method that uses the API collections to print the contents of a collection (`Collection`) in reverse order (of that which is covered by its iterator). (Unlike the previous problem, in this case do not assume that the input object is a `List`, and you only need to print the information in the order)

7. Build a method that removes all elements in pairs position of an entry list (`List`). Knowing that this list can either be of type `ArrayList` or `LinkedList` type and assuming that the memory allocation is done in constant time:

- a) It is intended that the extra space is used by the method (1). Develop the corresponding method.
- b) It is intended that the running time is $O(N)$. Develop the corresponding method.

8 – Create a structure that supports push () operations, pop () and findMin () in constant time ($O(1)$).. Implement this structure. (Hint: Use two stacks)